



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FCFM



FACULTAD DE CIENCIAS FÍSICO MATEMÁTICAS

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE CIENCIAS FÍSICO MATEMÁTICAS

Producto Integrador de Aprendizaje Reporte Técnico Final

Programación Avanzada para Ciberseguridad

2007817 Alejandra Itzel Ojeda Valdez *Github: imaleim*

2019637 Joustin Joel Martinez Carranza *Github: Joustin21*

2063816 Luis Santiago Cantu Niño *Github: sqntii*

A 20 de mayo de 2026

Tabla de contenido

Resumen	3
Diseño	4
Estructura general del proyecto.....	4
Diseño de módulos.....	5
file_reader.....	5
hasher	5
process_enum	6
packet_sniffer.....	6
Implementación	7
Herramientas utilizadas.....	7
Compilación	7
Pruebas realizadas.....	8
Prueba de lectura de archivos.....	8
Prueba de hashing SHA-256.....	9
Prueba de enumeración de procesos	10
Prueba de captura ICMP	11
Análisis estático.....	12
Análisis dinámico.....	13
Ingeniería inversa	14
Riesgos y mitigaciones.....	15
Conclusiones	16

Resumen

SimulatorPIA es un componente educativo de ciberseguridad desarrollado en C++ cuyo objetivo principal es permitir el aprendizaje práctico de técnicas relacionadas con análisis defensivo, monitoreo del sistema, análisis estático, análisis dinámico y reversing básico dentro de un entorno controlado.

El proyecto fue diseñado para ejecutarse exclusivamente dentro de una máquina virtual Ubuntu, evitando cualquier riesgo hacia el sistema anfitrión y manteniendo un enfoque completamente académico y defensivo.

La herramienta implementa una arquitectura modular compuesta por diferentes componentes independientes que permiten realizar tareas relacionadas con:

- lectura de archivos locales
- generación de hashes SHA-256
- enumeración de procesos activos
- captura y análisis de tráfico ICMP
- análisis básico de binarios ELF

Durante el desarrollo también se trabajó con compilaciones debug y release para comparar diferencias relacionadas con símbolos de depuración y facilitar prácticas de reversing utilizando herramientas como strings y radare2.

El proyecto permitió comprender cómo múltiples capacidades aparentemente simples pueden integrarse dentro de un solo binario modular y cómo dichas capacidades pueden analizarse desde una perspectiva defensiva.

Diseño

SimulatorPIA fue diseñado utilizando una arquitectura modular en C++ donde cada funcionalidad principal se encuentra separada en módulos independientes dentro del directorio /src.

Esta estructura permite mantener el código organizado, facilitar futuras modificaciones y mejorar la comprensión individual de cada componente.

El programa principal (main.cpp) funciona como coordinador general del sistema y permite ejecutar los distintos módulos mediante un menú interactivo.

La arquitectura modular fue seleccionada debido a que facilita:

- mantenimiento del código
- escalabilidad
- separación lógica de funcionalidades
- análisis independiente de módulos
- pruebas controladas

Estructura general del proyecto

El proyecto se organiza utilizando múltiples directorios:

- /src contiene el código fuente
- /bin almacena los ejecutables generados
- /docs contiene documentación técnica
- /analysis incluye resultados de reversing
- /evidence almacena capturas y evidencias

Esta estructura busca mantener separación clara entre desarrollo, documentación y análisis.

Diseño de módulos

file_reader

El módulo `file_reader` fue diseñado para simular acceso controlado a archivos locales dentro del entorno virtual.

La funcionalidad principal consiste en abrir el archivo `test.txt`, leer su contenido línea por línea y mostrarlo en pantalla.

Aunque esta funcionalidad parece sencilla, representa un comportamiento común observado durante análisis dinámicos de programas y malware, donde el acceso a archivos puede indicar procesos de configuración, lectura de logs o manipulación de información.

El módulo utiliza:

- `ifstream`
- manejo básico de archivos
- lectura secuencial

También incluye validación simple para detectar errores de apertura.

hasher

El módulo `hasher` implementa generación de hashes SHA-256 reales utilizando OpenSSL.

Inicialmente el proyecto utilizaba hashes simulados, pero posteriormente fue actualizado para incorporar funciones criptográficas reales mediante:

- `SHA256_Init`
- `SHA256_Update`
- `SHA256_Final`

El objetivo de este módulo es demostrar cómo puede verificarse la integridad de archivos y cómo los hashes pueden identificarse posteriormente durante análisis estático.

El uso de SHA-256 también permitió introducir dependencias adicionales como:

- `libssl`
- `libcrypto`

La implementación realiza lectura del archivo por bloques y posteriormente genera el hash final en formato hexadecimal.

process_enum

El módulo process_enum permite enumerar procesos activos dentro del sistema Linux.

La implementación utiliza acceso al directorio /proc, el cual contiene información del sistema y procesos activos.

El módulo identifica directorios numéricos correspondientes a PIDs y posteriormente obtiene el nombre de cada proceso leyendo:

/proc/PID/comm

Esta funcionalidad representa una técnica común de monitoreo del sistema y análisis de comportamiento.

El módulo muestra:

- PID
- nombre del proceso

permitiendo observar actividad real dentro de la máquina virtual.

packet_sniffer

El módulo packet_sniffer implementa captura básica de tráfico de red utilizando la librería libpcap.

La herramienta detecta paquetes ICMP generados dentro de la red virtual y muestra:

- IP origen
- IP destino
- tamaño del paquete

La captura se realiza mediante:

- pcap_open_live
- pcap_loop
- pcap_findalldevs

El tráfico utilizado durante las pruebas fue generado manualmente mediante comandos ping.

El módulo fue diseñado para ejecutarse únicamente dentro del entorno virtual y con privilegios sudo.

Implementación

El desarrollo del proyecto fue realizado completamente en C++ utilizando Ubuntu Linux dentro de una máquina virtual.

Herramientas utilizadas

Durante el desarrollo se utilizaron:

- g++
- libpcap
- OpenSSL
- strings
- rabin2
- VMware Workstation

Compilación

Se generaron dos versiones principales del ejecutable.

- **Versión debug**

La versión debug fue utilizada para análisis estático y reversing debido a que conserva símbolos de depuración.

Compilación utilizada:

```
g++ src/*.cpp -lpcap -lssl -lcrypto -g -o bin/SimulatorPIA_debug
```

- **Versión release**

La versión release fue utilizada para comparar diferencias relacionadas con la eliminación de símbolos y metadata.

Compilación utilizada:

```
g++ src/*.cpp -lpcap -lssl -lcrypto -o bin/SimulatorPIA_release  
strip bin/SimulatorPIA_release
```

La herramienta strip permitió reducir información visible dentro del binario.

Pruebas realizadas

Todas las pruebas fueron ejecutadas dentro de una máquina virtual Ubuntu aislada.

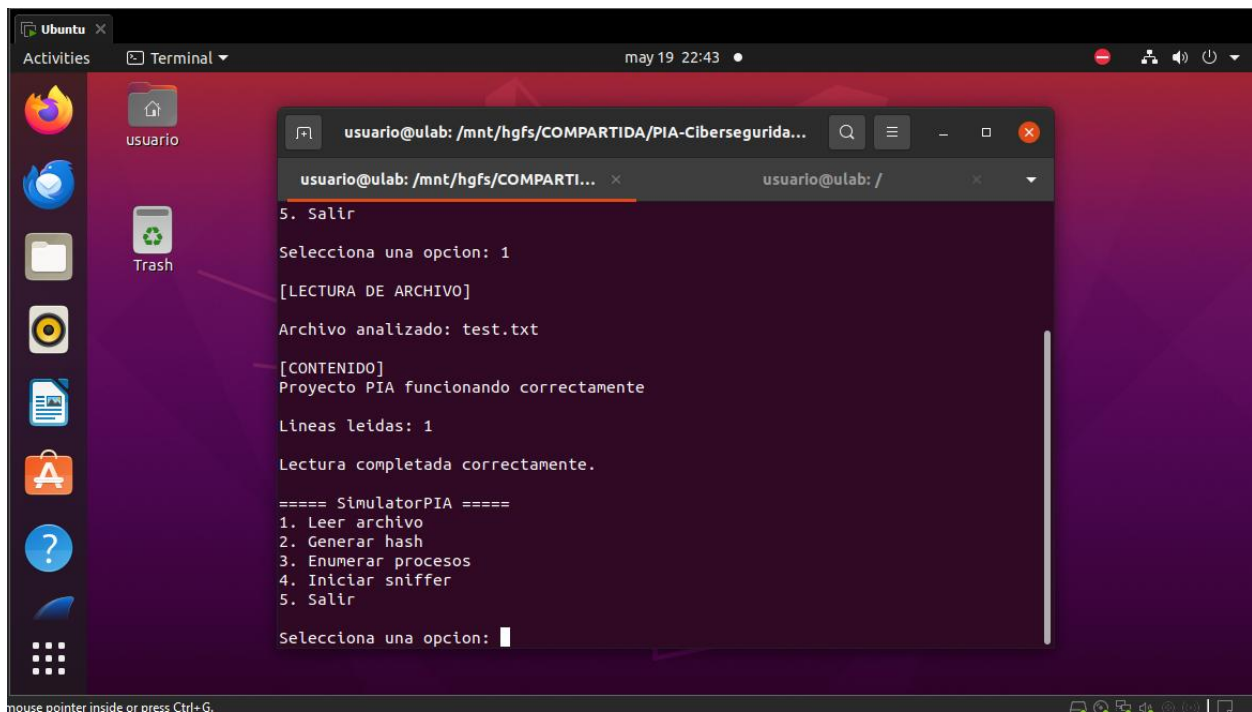
Prueba de lectura de archivos

Se verificó que el módulo file_reader lograra abrir correctamente test.txt y mostrar su contenido.

Resultado observado:

Proyecto PIA funcionando correctamente

La lectura fue realizada exitosamente sin errores.



The screenshot shows a terminal window titled 'usuario@ulab: /mnt/hgfs/COMPARTIDA/PIA-Cibersegurida...' with the following output:

```
5. Salir
Selecciona una opcion: 1
[LECTURA DE ARCHIVO]
Archivo analizado: test.txt
[CONTENIDO]
Proyecto PIA funcionando correctamente
Lineas leidas: 1
Lectura completada correctamente.

===== SimulatorPIA =====
1. Leer archivo
2. Generar hash
3. Enumerar procesos
4. Iniciar sniffer
5. Salir
Selecciona una opcion: 
```

Below the terminal window, the text 'mouse pointer inside or press Ctrl+G' is visible.

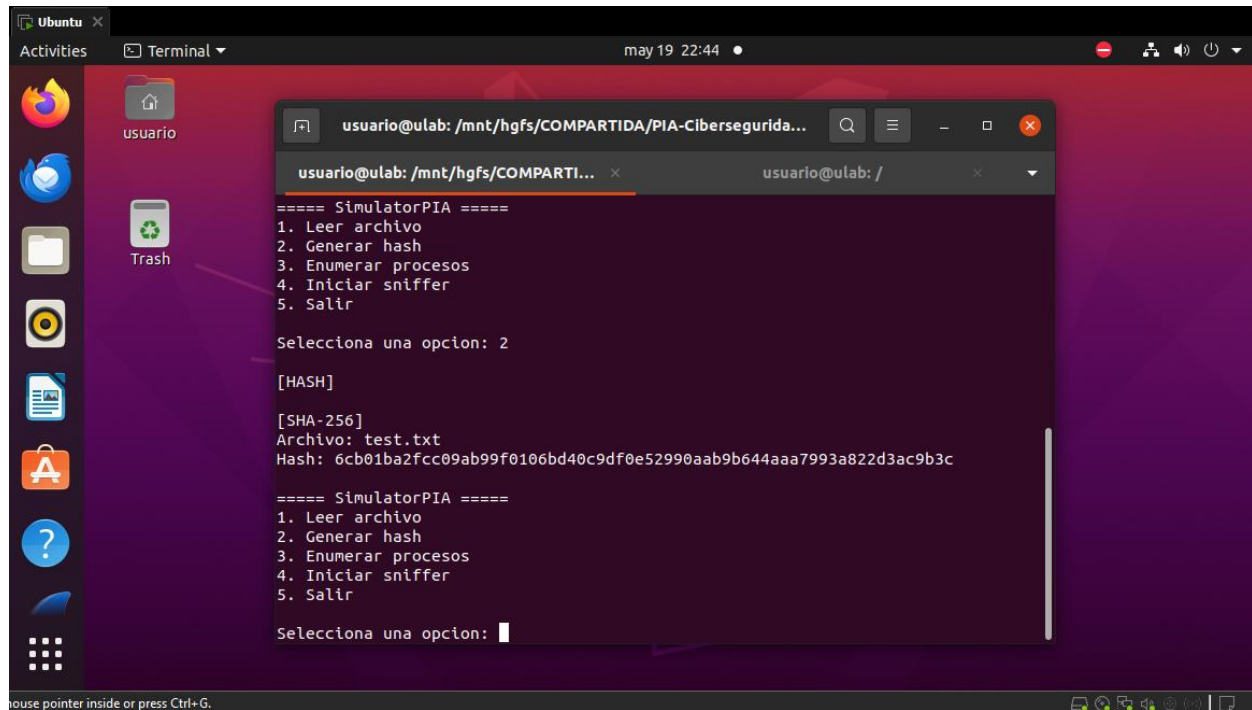
Prueba de hashing SHA-256

El módulo hasher fue probado utilizando el mismo archivo de prueba.

El sistema generó correctamente un hash SHA-256 válido mostrando:

- nombre del archivo
- hash hexadecimal

Esto permitió verificar el correcto funcionamiento de OpenSSL.



```
usuario@ulab: /mnt/hgfs/COMPARTIDA/PIA-Cibersegurida...
usuario@ulab: /mnt/hgfs/COMPARTI...
===== SimulatorPIA =====
1. Leer archivo
2. Generar hash
3. Enumerar procesos
4. Iniciar sniffer
5. Salir

Selecciona una opcion: 2

[HASH]

[SHA-256]
Archivo: test.txt
Hash: 6cb01ba2fcc09ab99f0106bd40c9df0e52990aab9b644aaa7993a822d3ac9b3c

===== SimulatorPIA =====
1. Leer archivo
2. Generar hash
3. Enumerar procesos
4. Iniciar sniffer
5. Salir

Selecciona una opcion: 
```

Prueba de enumeración de procesos

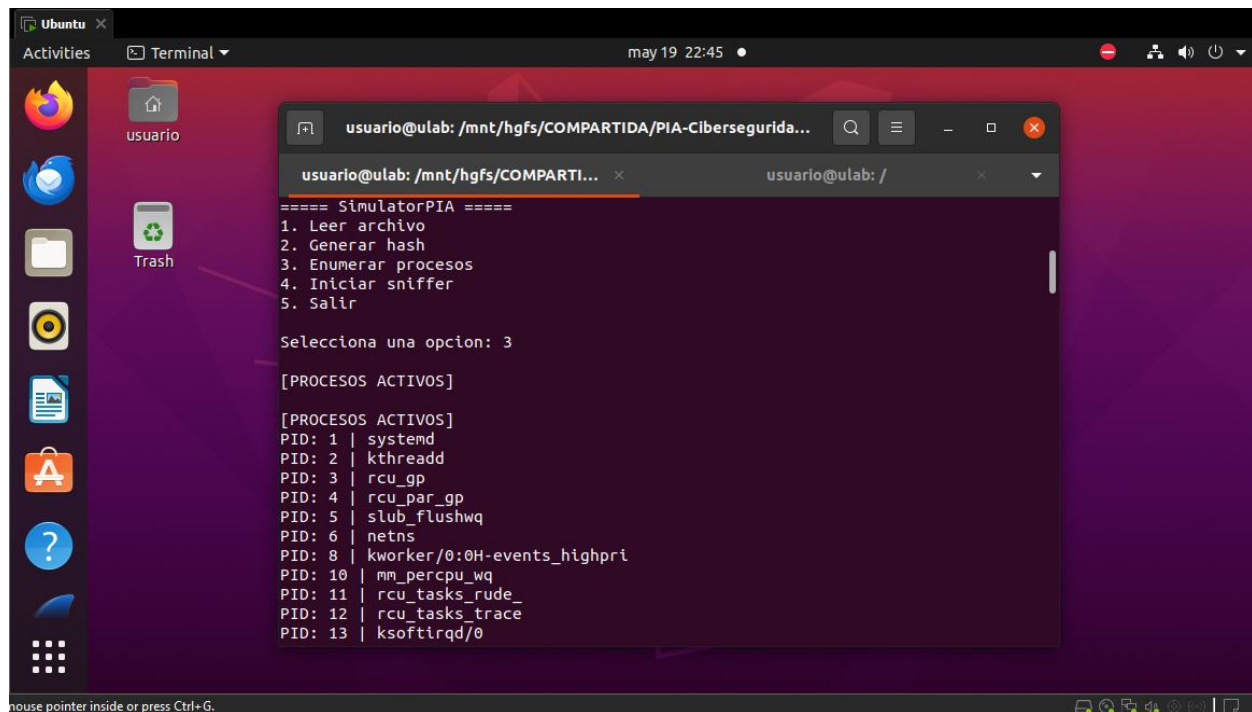
El módulo process_enum fue ejecutado para identificar procesos activos dentro del sistema Linux.

Se observó correctamente:

- PID
- nombres de procesos reales

incluyendo procesos del sistema como:

- systemd
- bash
- procesos internos del kernel



```
usuario@ulab: /mnt/hgfs/COMPARTIDA/PIA-Cibersegurida...
usuario@ulab: /mnt/hgfs/COMPARTI...
===== SimulatorPIA =====
1. Leer archivo
2. Generar hash
3. Enumerar procesos
4. Iniciar sniffer
5. Salir

Selecciona una opcion: 3

[PROCESOS ACTIVOS]

[PROCESOS ACTIVOS]
PID: 1 | systemd
PID: 2 | kthreadd
PID: 3 | rcu_gp
PID: 4 | rcu_par_gp
PID: 5 | slub_flushwq
PID: 6 | netns
PID: 8 | kworker/0:0H-events_highpri
PID: 10 | mm_percpu_wq
PID: 11 | rcu_tasks_rude_
PID: 12 | rcu_tasks_trace
PID: 13 | ksoftirqd/0
```

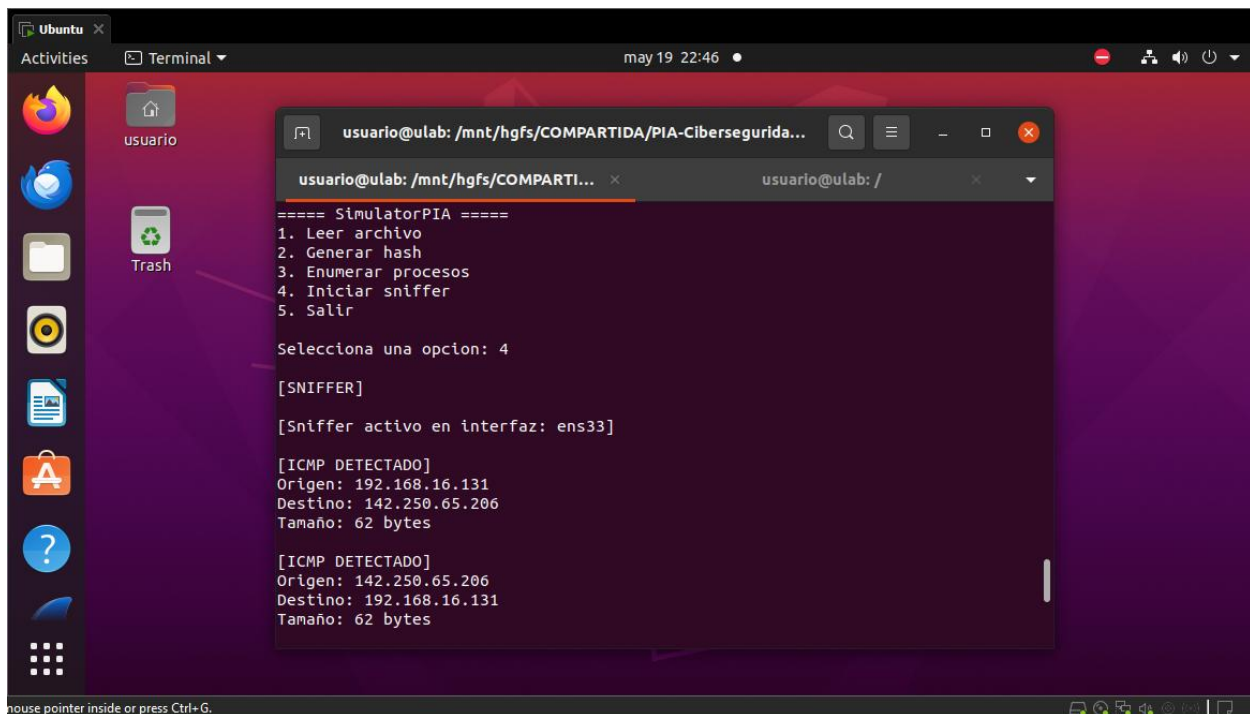
Prueba de captura ICMP

El módulo packet_sniffer fue probado generando tráfico ICMP mediante:
ping google.com

Durante la captura el sistema mostró correctamente:

- IP origen
- IP destino
- tamaño de paquetes

La detección se realizó en tiempo real utilizando libpcap.



The screenshot shows an Ubuntu desktop with a terminal window open. The terminal displays the output of a program called 'SimulatorPIA'. The program has a menu with five options: 1. Leer archivo, 2. Generar hash, 3. Enumerar procesos, 4. Iniciar sniffer, and 5. Salir. Option 4 was selected, and the program entered a sniffer mode. It then detected two ICMP packets. The first packet has an origin of 192.168.16.131 and a destination of 142.250.65.206, with a size of 62 bytes. The second packet has an origin of 142.250.65.206 and a destination of 192.168.16.131, also with a size of 62 bytes. The terminal window is titled 'usuario@ulab: /mnt/hgfs/COMPARTIDA/PIA-Cibersegurida...' and the desktop background is the Ubuntu logo.

```
usuario@ulab: /mnt/hgfs/COMPARTIDA/PIA-Cibersegurida...  
==== SimulatorPIA ====  
1. Leer archivo  
2. Generar hash  
3. Enumerar procesos  
4. Iniciar sniffer  
5. Salir  
  
Selecciona una opcion: 4  
  
[SNIFFER]  
  
[Sniffer activo en interfaz: ens33]  
  
[ICMP DETECTADO]  
Origen: 192.168.16.131  
Destino: 142.250.65.206  
Tamaño: 62 bytes  
  
[ICMP DETECTADO]  
Origen: 142.250.65.206  
Destino: 192.168.16.131  
Tamaño: 62 bytes
```

Análisis estático

El análisis estático fue realizado principalmente sobre la versión debug del binario.

Las herramientas utilizadas incluyeron:

- strings
- rabin2
- análisis ELF

Strings identificados

Durante el análisis se identificaron cadenas relacionadas con:

- libpcap
- OpenSSL
- SHA256
- tráfico ICMP
- menú interactivo
- funciones del sistema

También se detectaron mensajes internos generados por el programa.

Imports relevantes

El análisis de imports permitió identificar funciones importantes como:

- pcap_loop
- pcap_open_live
- pcap_findalldevs
- SHA256_Init
- SHA256_Update
- SHA256_Final
- opendir
- readdir

Estas funciones permiten inferir capacidades relacionadas con:

- monitoreo de red
- acceso al sistema
- generación de hashes
- enumeración de procesos

Secciones ELF

El análisis ELF permitió identificar secciones importantes como:

- .text
- .data
- .bss
- .rodata
- .symtab
- .strtab

La versión debug también incluyó múltiples secciones .debug_*.

Análisis dinámico

El análisis dinámico se enfocó en observar el comportamiento del programa durante su ejecución.

Las pruebas fueron realizadas dentro de la máquina virtual Ubuntu utilizando privilegios sudo.

Durante la ejecución se observaron comportamientos relacionados con:

- lectura de archivos
- captura de tráfico
- monitoreo de procesos
- generación de hashes

El análisis dinámico permitió confirmar que:

- no existe persistencia
- no hay modificaciones al sistema anfitrión
- no existe comunicación remota
- no se realizan acciones ofensivas reales

Todo el comportamiento observado fue consistente con los objetivos educativos del proyecto.

Ingeniería inversa

Se realizaron actividades básicas de reversing sobre el binario generado.

El objetivo fue identificar funciones relevantes, imports y capacidades del sistema utilizando herramientas simples de análisis.

Durante el reversing se detectaron claramente referencias a:

- libpcap
- OpenSSL
- funciones SHA256
- funciones relacionadas con procesos Linux

La comparación entre las versiones debug y release permitió comprender cómo la eliminación de símbolos afecta el análisis estático.

La versión debug facilitó considerablemente la identificación de funciones y estructuras internas debido a la presencia de símbolos y metadata adicional.

Por otro lado, la versión release dificultó parcialmente el análisis debido al uso de strip.

Riesgos y mitigaciones

Aunque SimulatorPIA es un proyecto educativo, algunas de las técnicas implementadas podrían parecer similares a comportamientos observados en herramientas ofensivas o malware real.

Las funcionalidades más sensibles incluyen:

- sniffing de tráfico
- monitoreo de procesos
- acceso al sistema
- análisis de comportamiento

Para reducir riesgos se aplicaron múltiples medidas:

- ejecución únicamente en VM
- ausencia de persistencia
- ausencia de comunicación remota
- ausencia de exfiltración
- ausencia de modificaciones críticas
- enfoque exclusivamente académico

El proyecto fue diseñado únicamente para fines educativos relacionados con análisis defensivo y reversing básico.

Conclusiones

SimulatorPIA permitió integrar múltiples conceptos relacionados con análisis defensivo, reversing y monitoreo de sistemas dentro de una sola aplicación modular.

El proyecto facilitó la comprensión práctica de:

- binarios ELF
- análisis estático
- análisis dinámico
- tráfico ICMP
- generación de hashes SHA-256
- monitoreo de procesos Linux

También permitió trabajar con herramientas utilizadas comúnmente en reversing y análisis defensivo como:

- strings
- rabin2
- OpenSSL
- libpcap

El uso de versiones debug y release permitió observar diferencias importantes relacionadas con símbolos de depuración y metadata interna.

Finalmente, el proyecto reforzó buenas prácticas relacionadas con:

- modularidad en C++
- documentación técnica
- análisis controlado
- ejecución segura dentro de máquinas virtuales

SimulatorPIA cumplió exitosamente los objetivos académicos planteados y permitió desarrollar habilidades prácticas relacionadas con análisis defensivo y reversing básico dentro de un entorno controlado.